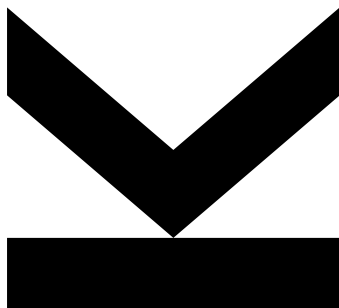


Simon Grundner
Institute for Signal
Processing

@ k12136610@students.jku.at
🌐 <https://jku.at/isp>

October 3, 2025

03 - Pulse Width Modulation



Hardwaredesign PR - Exercise 03

Semester 2024W

Contents

1. Pulse Width Modulation	2
1.1 Model Implementation	3
1.2 Simulation	5

1. Pulse Width Modulation

Pulse Width Modulation

Figure 1: Pulse Width Modulation

In this exercise, a pulse width modulation (PWM) generator should be implemented. An example PWM signal is schematically shown in Figure 1. There the two important parameters period (time) and on time is shown. These two parameters define the so-called duty cycle: on time divided by period. The PWM generator will be used for two tasks in this course: to control a servo motor and to build a digital to analog converter (DAC). For these applications (more details will follow in later exercises), the output level (either voltage or position of the servo) is defined via the duty cycle. Use the following entity for the top-level design:

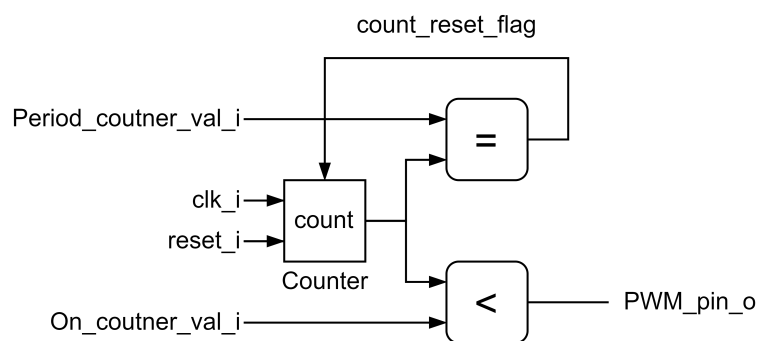
```
entity PWM is
  generic (
    COUNTER_LEN : natural
  );
  port (
    clk_i          : in std_ulogic;
    reset_i        : in std_ulogic;
    Period_counter_val_i : in unsigned (COUNTER_LEN-1 downto 0);
    ON_counter_val_i   : in unsigned (COUNTER_LEN-1 downto 0);
    PWM_pin_o       : out std_ulogic
  );
end entity PWM;
```

Why do you think the counter values for the period and the one-time are specified as inputs?

To-Do List

- Draw a block diagram of the architecture of the PWM. Do you need to have a finite state machine? If you do, please write a state diagram.
- Write the entities and architectures that are required in VHDL. Use one .vhd file for each entity and one .vhd file for each architecture.
- Write a testbench in VHDL (extra file). Simulate the system using Modelsim and hand in screenshots showing the output of the waveform window. Find meaningful test cases. Please also include the test cases with a duty cycle of 0% (the output signal has to stay at 0 over the periods for this test case) and a duty cycle of 100% (the output signal has to stay at 1 over the periods for this test case).
- Discuss the results of the simulation in a few sentences and answer the questions.

1.1 Model Implementation



The PWM generator uses a counter to count up to the specified period. When the counter value is less than the duty cycle, the output signal is high and otherwise low. The counter resets when it reaches the period value. As the counter only consist of one register process as seen in EX02 and the PWM module only adds combinatorics to the output, no state-machine is needed.

The *Counter* is implemented as following process:

```

1  cnt_reg : process (rst_i, clk_i) is
2  begin
3      if rst_i = '1' then
4          count <= (others => '0');
5      elsif rising_edge(clk_i) then
6          count <= count + 1;
7          if count_reset_flag = '1' then
8              count <= (others => '0');
9          end if;
10     end if;
11 end process cnt_reg;

```

Listing 1: Counter process

The “<”-Block is implemented as following process:

```
1  pwm_comb : process (count, ON_counter_val_i) is
2  begin
3      if count < ON_counter_val_i then
4          PWM_pin_o <= '1';
5      else
6          PWM_pin_o <= '0';
7      end if;
8  end process pwm_comb;
```

Listing 2: PWM Output process

The “=”-Block is implemented as following process:

```
1  sync_reset_comb : process (count, Period_counter_val_i) is
2  begin
3      if count = Period_counter_val_i-1 then
4          count_reset_flag <= '1';
5      else
6          count_reset_flag <= '0';
7      end if;
8  end process sync_reset_comb;
```

Listing 3: Counter Reset process

1.2 Simulation

Waveforms of various period and duty cycle values are shown below.

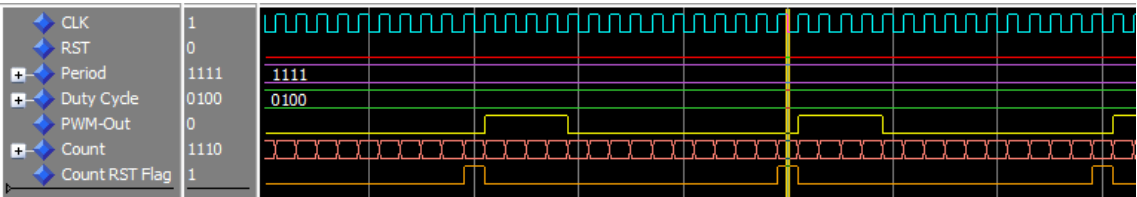


Figure 2: Behaviour with Period 15 and Duty Cycle 4

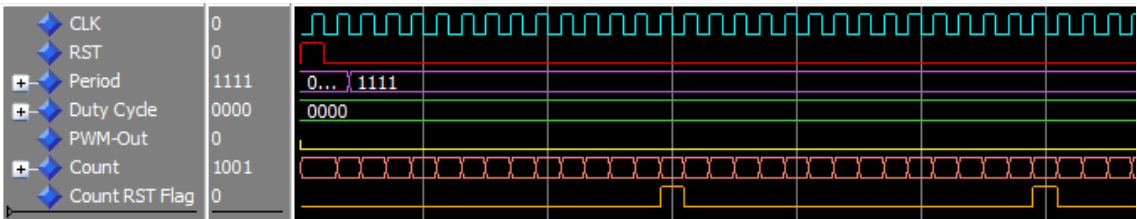


Figure 3: Behaviour with Period 15 and Duty Cycle 8

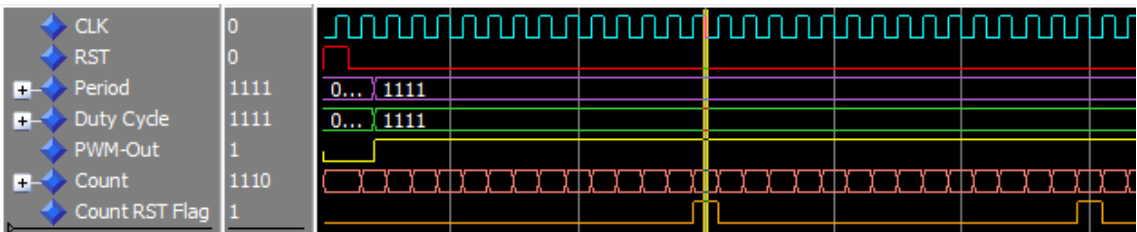


Figure 4: Behaviour with Period 15 and Duty Cycle 15